Deriving Scaling Laws for a Small Greek Language Model

1. Objective

This project studies how validation loss changes as model size, training-token budget, and total compute scale in a small Greek-language language-model setting. The goal was to build a fully reproducible pretraining pipeline, run a structured sweep over model size and token budget, and fit empirical scaling relationships inspired by prior scaling-law literature.

The study was conducted on the Greek TinyStories dataset (`alexliap/tinystories-gr`) using a tokenizer trained from scratch on Greek-only text. All models were trained from random initialization and evaluated using final validation loss.

---

2. Dataset and Preprocessing

We used the Hugging Face dataset `alexliap/tinystories-gr` and retained only the `greek_translation` field as the training corpus. No English text was used at any stage of tokenizer training, data preparation, or model training.

Preprocessing Policy

The preprocessing logic was intentionally kept simple and consistent with the tokenizer pipeline. Each example was processed as follows:

1. keep only entries that are valid strings,
2. apply `strip()` to remove leading and trailing whitespace,
3. discard empty entries.

This ensured that the text seen during data preparation matched the assumptions used when training the tokenizer, avoiding train-time inconsistencies between tokenization and downstream model inputs.

Train/Validation Split

The dataset was loaded from the Hugging Face `train` split and then shuffled with a fixed random seed (`seed = 42`). A **row-level** split was used to create the final train and validation sets:

- **training fraction:** 99%
- **validation fraction:** 1%

Importantly, the split was performed at the level of complete stories rather than at the token level, so that validation examples remained disjoint from training examples.

The final split sizes were:

- **training texts:** 2,120,004
- **validation texts:** 21,414

Frozen Tokenizer Reuse

Data preparation reused the frozen tokenizer selected in the tokenizer stage:

- **Tokenizer file:** `artifacts/tokenizer/final_vocab_8000/tokenizer.json`

No new tokenizer was trained during this stage. This ensured that all experiments used the exact same tokenization scheme and vocabulary of **8,000** tokens.

Tokenization and Sequence Construction

Each story was encoded with the frozen tokenizer into token IDs. Because the tokenizer includes post-processing with boundary markers, each encoded story automatically contains sequence boundary tokens such as `[BOS]` and `[EOS]`.

After tokenization, all token IDs within each split were concatenated into one continuous stream:

- one stream for the training split,
- one stream for the validation split.

These streams were then segmented into fixed-length chunks of:

- **context length = 128 tokens**

This chunking strategy produced dense fixed-shape arrays for causal language modeling. Any final incomplete remainder shorter than 128 tokens was dropped.

Final Tokenized Dataset Statistics

The final tokenized outputs were:

- **training tokens:** 482,438,836
- **validation tokens:** 4,879,001
- **training chunks:** 3,769,053
- **validation chunks:** 38,117

The resulting tensor shapes were:

- **train:** `(3,769,053, 128)`
- **validation:** `(38,117, 128)`

The number of discarded remainder tokens was negligible:

- **training dropped tokens:** 52
- **validation dropped tokens:** 25

Saved Outputs

The prepared data stage produced the following runtime artifacts under `artifacts/data/`:

- `train_chunks.npy`
- `val_chunks.npy`
- `data_prep_metadata.json`
- `sample_batch_preview.json`

These files were then used directly by the training stage. In particular, the `.npy` chunk arrays served as the fixed tokenized inputs for all subsequent model runs in the experimental grid.

---

3. Tokenizer Design

We trained a single BPE tokenizer from scratch using only the `greek_translation` field of the TinyStories-GR training split. The tokenizer was then frozen and reused unchanged across all model runs so that vocabulary design would not become an additional confounding variable in the scaling-law analysis.

Design Goals

The vocabulary size was not chosen arbitrarily. Instead, we selected it through a small empirical profiling sweep guided by three competing objectives:

1. **Tokenization efficiency**
   If the vocabulary is too small, Greek words are split into too many subword units. This increases sequence length and reduces the amount of semantic content that fits into a fixed context window.

2. **Parameter efficiency in small models**
   In the tiny-model regime, the embedding table can consume a substantial fraction of the total parameter budget. Since our smallest model is heavily parameter-constrained, we explicitly tracked the relative embedding cost of each vocabulary candidate.

3. **Vocabulary utilization**
   TinyStories-GR is a domain-constrained corpus with relatively simple and repetitive language. If the vocabulary is too large, many learned tokens become rare, which reduces effective vocabulary utilization and allocates capacity to entries that receive few updates.

Literature-Guided Heuristics

Our profiling was informed by literature-inspired heuristics rather than strict hard constraints, drawing on prior work on multilingual tokenization and scaling behavior in small-model regimes (Rust et al., 2021; Kaplan et al., 2020). In particular:

- Following prior multilingual tokenization work, we treated **fertility** (tokens per word) as a proxy for segmentation quality in a morphologically richer language such as Greek.
- Following scaling-law work in small-model regimes, we tracked the **embedding share** to monitor whether vocabulary size was consuming too much of the parameter budget.
- We used approximate target ranges as **soft reference points**:
  - fertility roughly in the **1.35–1.60** range,
  - embedding share ideally in the **15%–25%** range for the smallest model.

These targets were used as guiding assumptions, not as strict pass/fail rules. The final decision was based on the overall empirical trade-off observed in the profiling curves and token frequency distribution.

Tokenizer Configuration

We used a BPE tokenizer with the following configuration:

- **training text**: Greek-only text from `greek_translation`
- **normalization**: Unicode `NFKC`
- **pre-tokenization**: whitespace-based splitting
- **special tokens**: `[PAD]`, `[UNK]`, `[BOS]`, `[EOS]`

We intentionally preserved Greek orthographic information by avoiding lowercasing and accent stripping.

Vocabulary Profiling Procedure

We profiled the candidate vocabulary sizes **4k, 8k, 12k, and 16k** on a representative subset of the training split. For each candidate, we computed:

- **Fertility**: average number of subword tokens per whitespace-delimited word
- **Embedding share**: estimated fraction of a small-model parameter budget allocated to token embeddings
- **Active vocabulary ratio**: fraction of vocabulary items that appear at least a minimum number of times in the profiling subset

The goal was to find the smallest vocabulary size after which fertility improvements started to show diminishing returns, while also monitoring embedding growth and avoiding excessive vocabulary sparsity.

Profiling Results and Figure Interpretation

The vocabulary profiling results supported a compromise around **8,000** tokens.

The **fertility curve** decreased as vocabulary size increased, confirming that larger vocabularies reduce subword fragmentation. In our profiling sweep, fertility dropped from **1.4168** at 4k to **1.2886** at 8k, then more modestly to **1.2440** at 12k and **1.2239** at 16k. This indicates diminishing returns beyond 8k: increasing the vocabulary further still improves segmentation efficiency, but the gains become progressively smaller.

The **embedding-share curve** increased approximately linearly with vocabulary size, showing that larger vocabularies impose a direct parameter-cost penalty. This was particularly relevant for the smallest-model regime, where vocabulary growth competes with transformer capacity for a limited parameter budget. For this reason, we did not treat larger vocabularies as “free” improvements, even when they slightly reduced fertility.

The **token frequency histogram** showed a clear long-tail distribution: a relatively small number of tokens appeared very frequently, while many tokens appeared rarely. This confirmed that the corpus is lexically simple but still contains many infrequent entries. As a result, excessively large vocabularies would allocate increasing capacity to sparse tokens that receive limited training signal.

An additional useful observation is that the fertility curve exhibits an **elbow** around **8k** vocabulary items. Moving from 4k to 8k gives a meaningful reduction in fragmentation, whereas the improvements from 8k to 12k and 16k are much smaller relative to the additional vocabulary cost. This elbow pattern is one of the main reasons we treated 8k as the most balanced choice.

Taken together, the profiling figures suggest the following trade-off:

- **4k** is more compact but fragments words more aggressively,
- **12k** and **16k** reduce fertility only marginally relative to 8k,
- **8k** offers the best overall balance between tokenization efficiency, embedding cost, and vocabulary utilization.

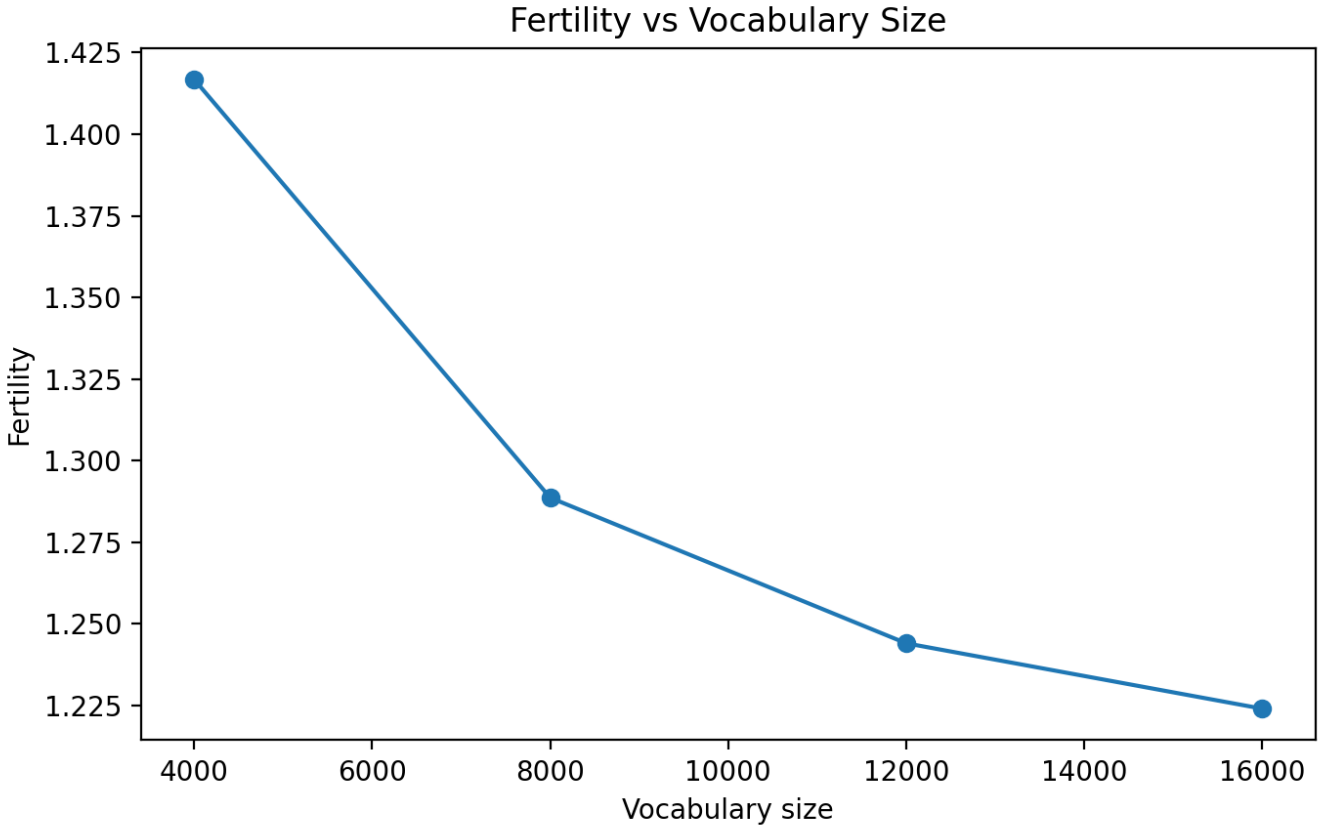
Final Selection

Based on this profiling process, we selected a vocabulary size of **8,000** tokens for all experiments. This choice was not derived from a single hard threshold, but from the overall empirical trade-off observed across the profiling metrics and plots. In practice, **8k** substantially reduced fragmentation relative to 4k, while avoiding the extra embedding cost and lower vocabulary utilization associated with 12k and 16k.

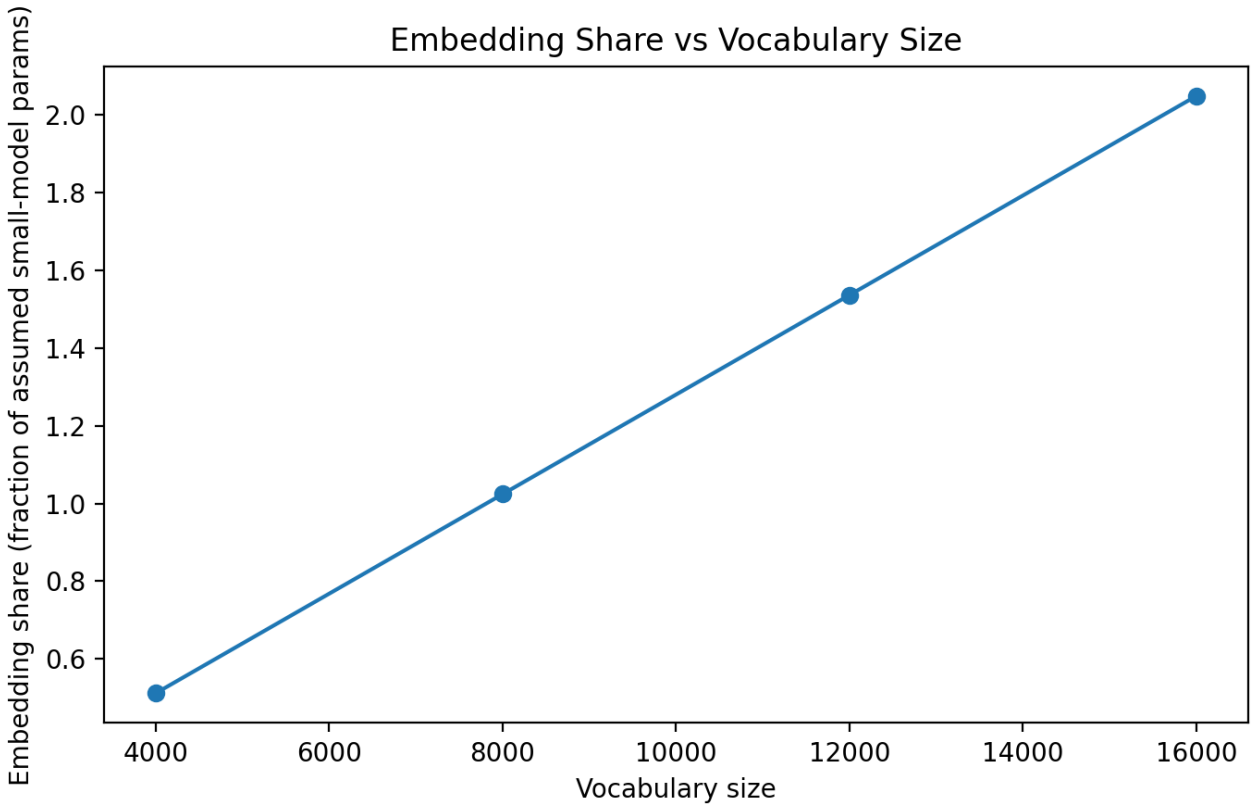
After selecting the final vocabulary size, we retrained the tokenizer once on the full training split and reused the saved tokenizer unchanged across the entire experimental grid. Detailed definitions of the tokenizer profiling metrics are provided in **Appendix B**.

Figures for Tokenizer Profiling

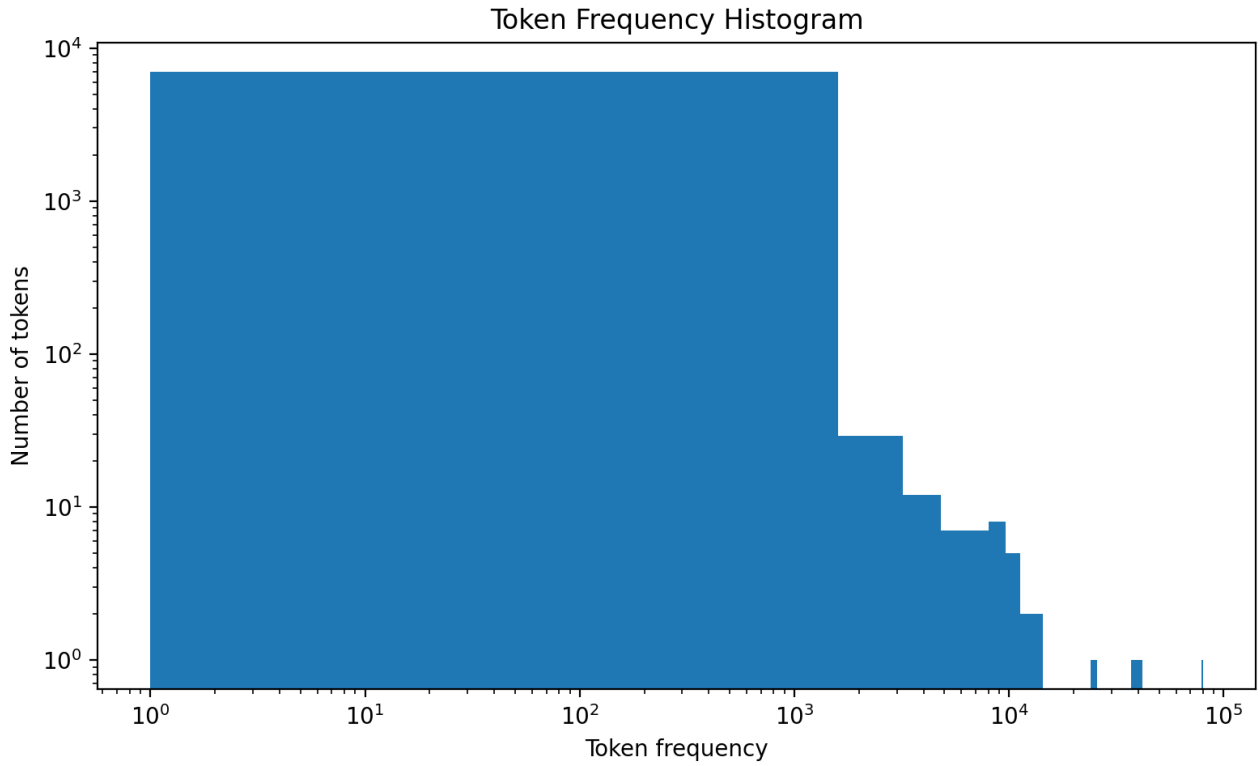
- **Figure 1.** Fertility vs vocabulary size
  Figure file: `reports/figures/fertility_vs_vocab_size.png`



- **Figure 2.** Embedding share vs vocabulary size
  Figure file: `reports/figures/embedding_share_vs_vocab_size.png`



- **Figure 3.** Token frequency histogram
  Figure file: `reports/figures/token_frequency_histogram.png`



---

4. Model Architecture and Configurations

All experiments used the same GPT-style **decoder-only causal language model** family, trained from random initialization. The architecture consisted of token embeddings, learned positional embeddings, a stack of Transformer decoder blocks with causal self-attention, and a final language-model head. We also used **tied input/output embeddings** in all model variants.

To isolate the effect of scale, we kept the following design choices fixed across the model family:

- vocabulary size: **8,000**
- context length: **128**
- MLP ratio: **4.0**
- dropout: **0.1**
- bias terms: **enabled**
- tied embeddings: **enabled**

The main scaling dimensions were therefore **depth** (`n_layer`) and **width** (`n_embd`), with the number of attention heads adjusted accordingly.

An important note is that the model names `model_1m`, `model_5m`, and `model_20m` are **nominal scale labels** rather than exact parameter counts. The actual trainable parameter counts measured from the implemented architecture are shown in Table 1.

Model name	Actual trainable parameters	Layers (`n_layer`)	Heads (`n_head`)	Embedding dim (`n_embd`)	Context length	MLP ratio	Dropout	Tied embeddings
model_1m	1,833,728	4	4	128	128	4.0	0.1	Yes
model_5m	5,451,264	6	8	224	128	4.0	0.1	Yes
model_20m	20,866,560	10	8	384	128	4.0	0.1	Yes

This model family was intentionally simple and consistent across runs. Rather than changing architectural components between experiments, we scaled a single decoder-only design across three sizes, which makes the observed loss trends easier to interpret in the scaling-law analysis.

Table

- **Table 1.** Model configurations
  Table file: `reports/tables/model_configs.csv`

A more detailed architecture-to-code mapping and explicit justification of the GPT-style decoder-only design are provided in **Appendix A**.

---

5. Experiment Grid and Training Setup

After validating the pipeline with a single vertical-slice training run, we executed a structured **3 × 3 experimental grid** spanning model size and training-token budget.

Fixed Training Setup

All sweep runs used the same core training setup:

- tokenizer: frozen 8,000-token Greek BPE tokenizer
- context length: **128**
- batch size: **16**
- learning rate: **3e-4**
- weight decay: **0.01**
- evaluation frequency: every **50** steps
- evaluation batches: **20**
- seed: **42**
- device: **CUDA**

Sweep Dimensions

The sweep varied two axes:

1. **Model size**
   - `model_1m`
   - `model_5m`
   - `model_20m`
2. **Training token budget**
   - **2M**
   - **10M**
   - **50M**

This produced a total of **9 runs**.

Step Budget Derivation

For each run, the number of optimizer steps was derived from the token budget as:

$$\text{tokens per step} = \text{batch size} \times (\text{context length} - 1)$$

With batch size = 16 and context length = 128:

$$\text{tokens per step} = 16 \times 127 = 2032$$

The step budget for each run was then computed as:

$$\text{max steps} = \left\lfloor \frac{\text{token budget}}{2032} \right\rfloor$$

This ensured that each run consumed approximately the intended number of training tokens while keeping the training protocol standardized.

Final Experiment Grid

Run name	Model	Trainable parameters	Token budget	Actual training tokens	Max steps	Estimated FLOPs
model_1m_tok_2000000	model_1m	1,833,728	2,000,000	1,999,488	984	21,999,102,787,584
model_1m_tok_10000000	model_1m	1,833,728	10,000,000	9,999,472	4,921	110,017,870,749,696
model_1m_tok_50000000	model_1m	1,833,728	50,000,000	49,999,392	24,606	550,111,710,560,256
model_5m_tok_2000000	model_5m	5,451,264	2,000,000	1,999,488	984	65,398,421,716,992
model_5m_tok_10000000	model_5m	5,451,264	10,000,000	9,999,472	4,921	327,058,570,395,648
model_5m_tok_50000000	model_5m	5,451,264	50,000,000	49,999,392	24,606	1,635,359,313,788,928
model_20m_tok_2000000	model_20m	20,866,560	2,000,000	1,999,488	984	250,334,617,927,680
model_20m_tok_10000000	model_20m	20,866,560	10,000,000	9,999,472	4,921	1,251,927,494,737,920
model_20m_tok_50000000	model_20m	20,866,560	50,000,000	49,999,392	24,606	6,259,891,878,789,120

The FLOP estimate was computed using the standard approximation:

$$\text{FLOPs} \approx 6 \times N \times D$$

where N is the number of trainable parameters and D is the number of training tokens actually consumed.

Table

- **Table 2.** Experiment grid
  Table file: `reports/tables/experiment_grid.csv`

---

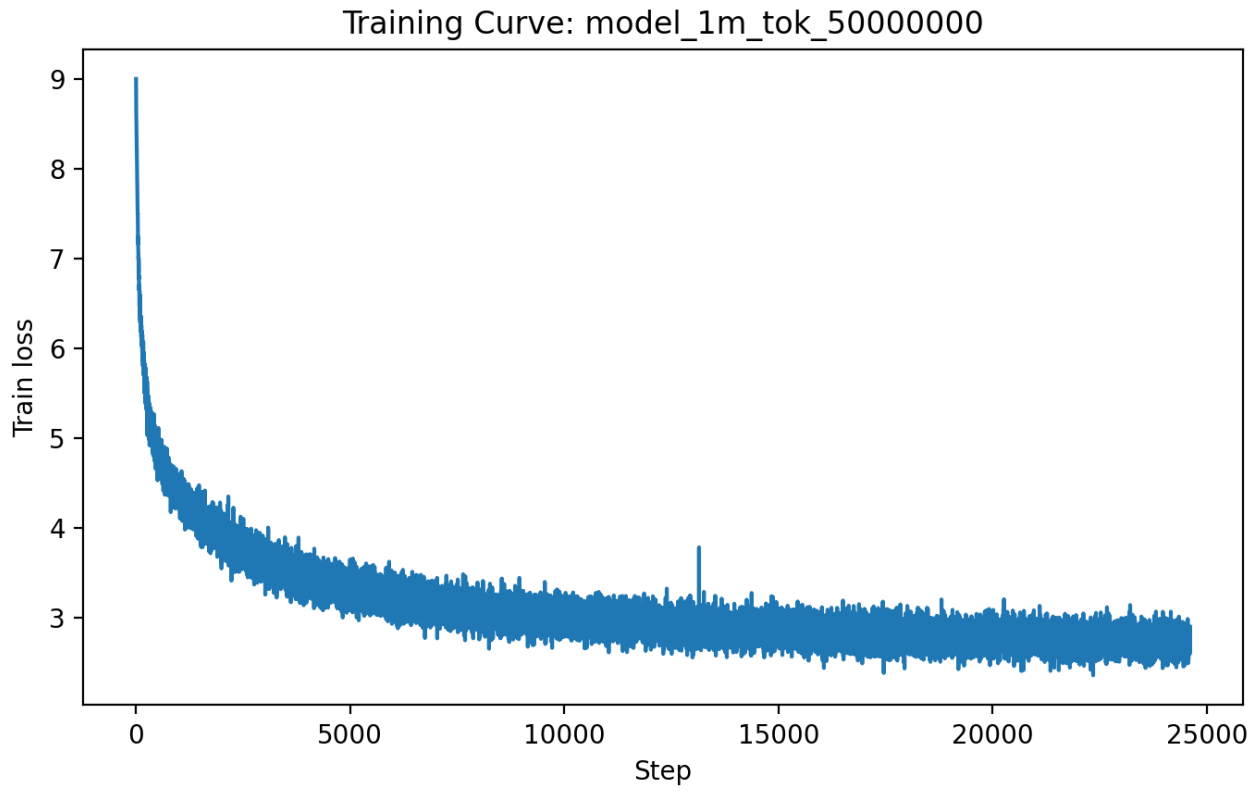
6. Training Curves

Training curves were generated for all 9 runs. In all cases, training loss decreased steadily over optimization steps, indicating stable optimization behavior across the full sweep. Runs with larger token budgets trained for more steps and reached lower final training losses. Larger models also generally achieved lower final training loss under the same token budget.

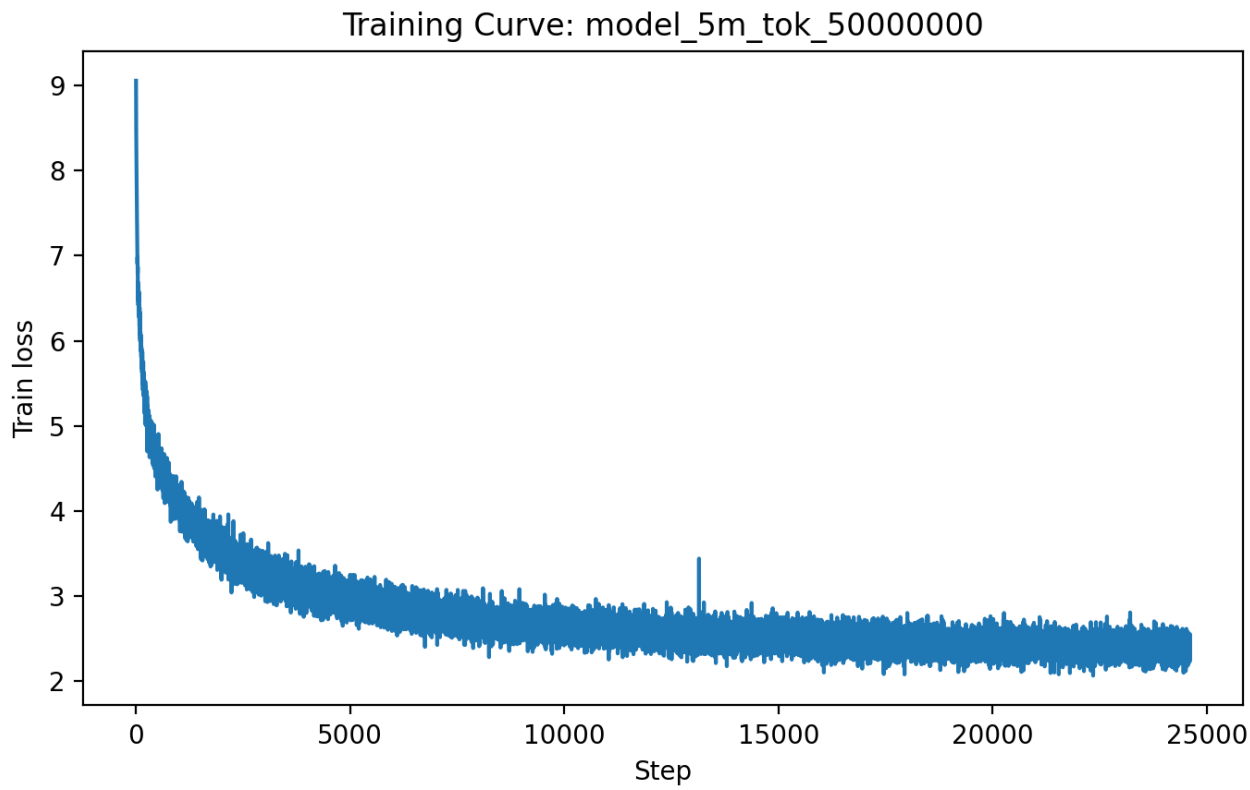
Representative curves for the three model sizes at the highest token budget are shown in the main text, while the full set of per-run curves is provided in the appendix / supplementary figures.

Representative Training Curves

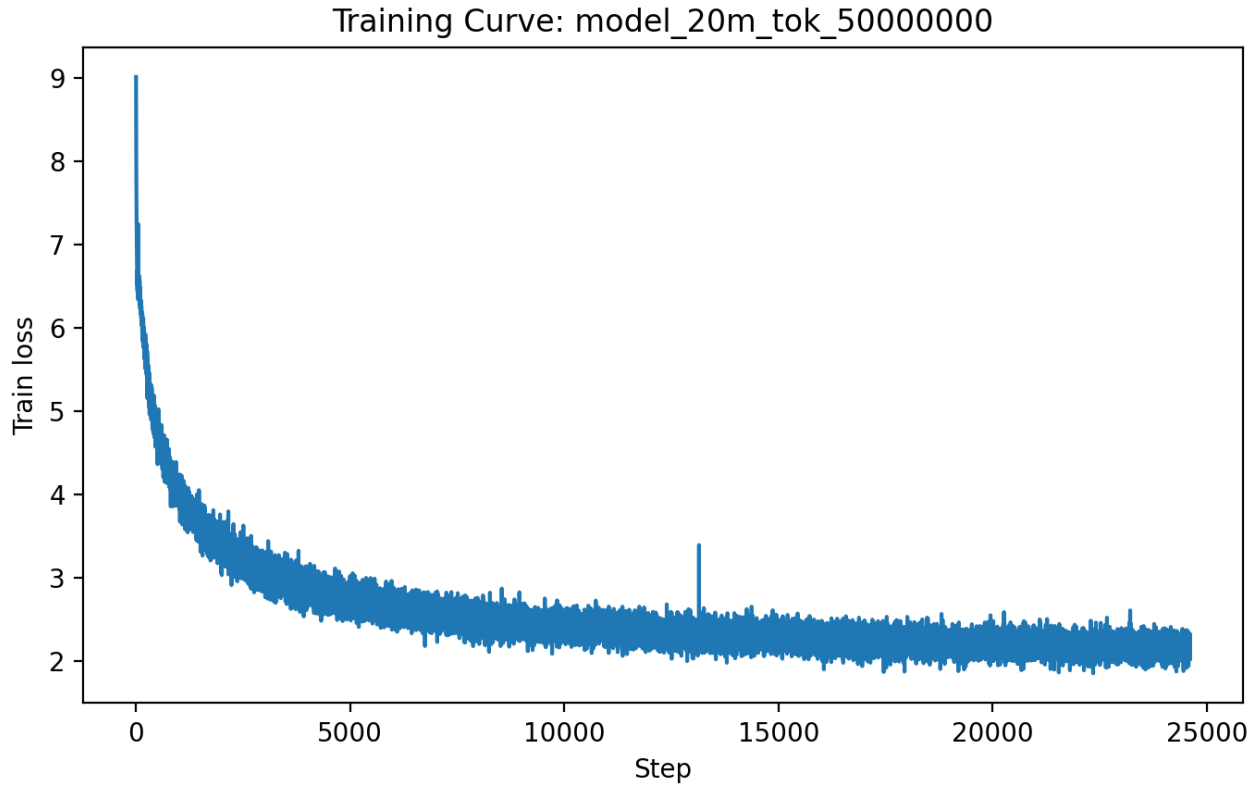
- **Figure 4.** Training curve for `model_1m_tok_50000000` **Figure file:** `reports/figures/training_curves/model_1m_tok_50000000_train_curve.png`



- **Figure 5.** Training curve for `model_5m_tok_50000000` **Figure file:** `reports/figures/training_curves/model_5m_tok_50000000_train_curve.png`



- **Figure 6.** Training curve for `model_20m_tok_50000000` **Figure file:** `reports/figures/training_curves/model_20m_tok_50000000_train_curve.png`



Supplementary Training Curves

- full combined view: **Figure file:** `reports/figures/all_training_curves.png`
- per-run training curves: **Figure folder:** `reports/figures/training_curves/`

Table

- **Table 3.** Training-curve summary **Table file:** `reports/tables/training_curves_summary.csv`

---

7. Results Overview

The final sweep consisted of 9 runs spanning three model sizes and three token budgets. The main result is that validation loss improved monotonically both with increased training tokens and with increased compute. Larger models also generally performed better at fixed token budgets, although the size-only trend was weaker when viewed through a single global 1D fit. Training curves across all runs showed stable optimization behavior, with monotonic loss reduction over steps and lower final train loss for larger token budgets and larger models.

Table

- **Table 4.** Final sweep results
  Table file: `reports/tables/results.csv`

---

8. Scaling Law Fits

After completing the 9-run experimental grid, we analyzed how validation loss scales with model size, training tokens, and total compute. We fitted simple one-dimensional power laws of the form:

[L(x) = a \cdot x^b]

where (L) is the final validation loss and (x) is one of:

- number of parameters (N),
- number of training tokens (D),
- estimated FLOPs.

All fits were performed in log-log space.

Loss vs Parameters

The fitted relation for validation loss as a function of parameter count was:

$$L(N) \approx 8.1233 \cdot N^{-0.0634}$$

with:

- exponent: **-0.0634**
- R^2 in log-space: **0.0610**

This fit is weak. Although larger models generally performed better at fixed token budgets, a single global 1D fit of loss versus parameter count did not explain much of the variance in the full 9-run grid. This is expected, because the parameter axis is confounded by simultaneous variation in training-token budget.

Loss vs Training Tokens

The fitted relation for validation loss as a function of training tokens was:

$$L(D) \approx 60.6623 \cdot D^{-0.1861}$$

with:

- exponent: **-0.1861**
- R^2 in log-space: **0.9176**

This was the strongest of the three 1D fits. It indicates a clear and consistent scaling trend with data: as the number of training tokens increases, validation loss decreases in a predictable power-law-like manner across the experiment grid.

Loss vs FLOPs

The fitted relation for validation loss as a function of estimated compute was:

$$L(C) \approx 345.2765 \cdot C^{-0.1414}$$

with:

- exponent: **-0.1414**
- R^2 in log-space: **0.8334**

This fit is also strong, showing that the final validation loss improves systematically as total compute increases.

Interpretation

Taken together, the fitted exponents show that the cleanest scaling signal in our experiments comes from **training tokens** and **compute**, while a simple global 1D fit with **parameter count alone** is much less informative.

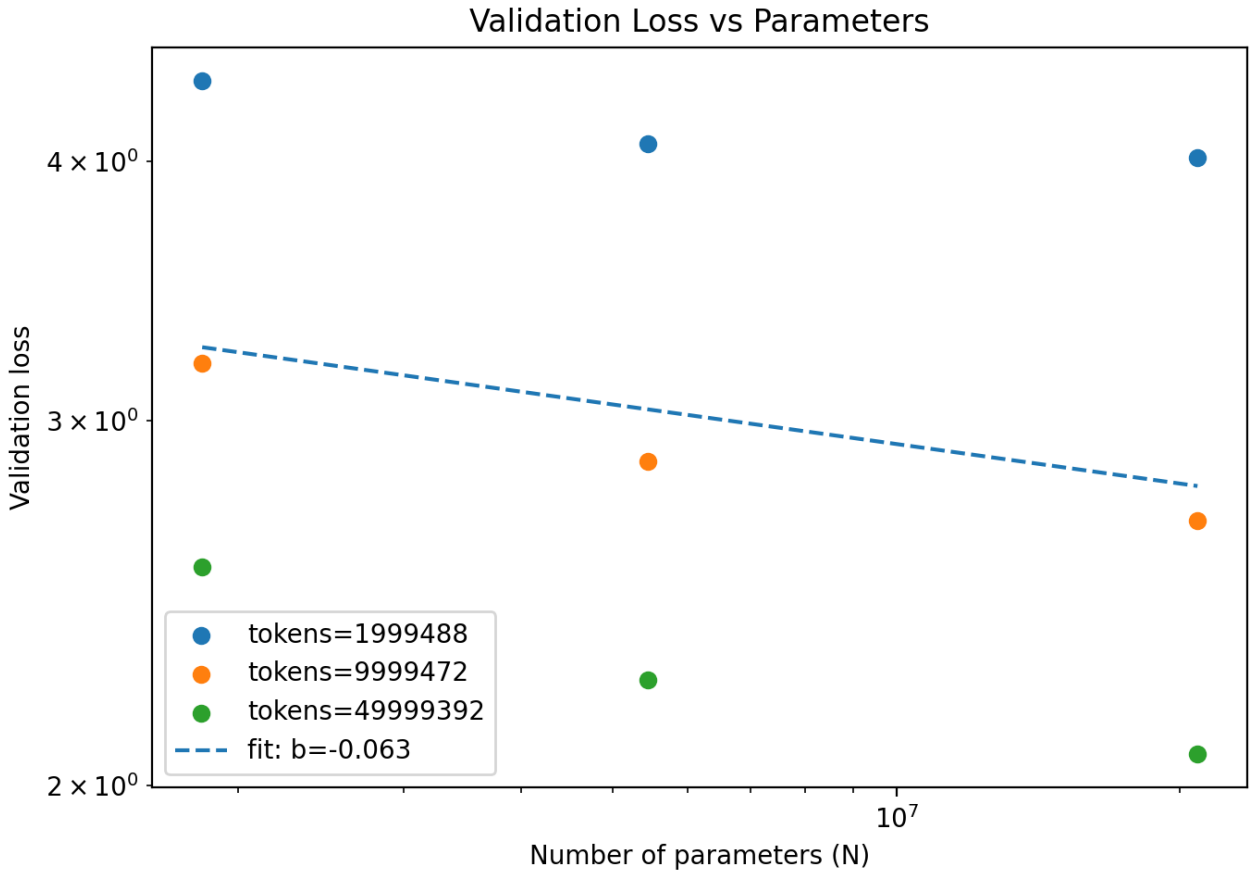
This does **not** mean that model size is unimportant. In the raw experiment grid, larger models usually outperform smaller ones at the same token budget. However, because both model size and token budget vary simultaneously across runs, the parameter-only fit is not sufficient to isolate that effect.

A practical interpretation of the fitted trends is:

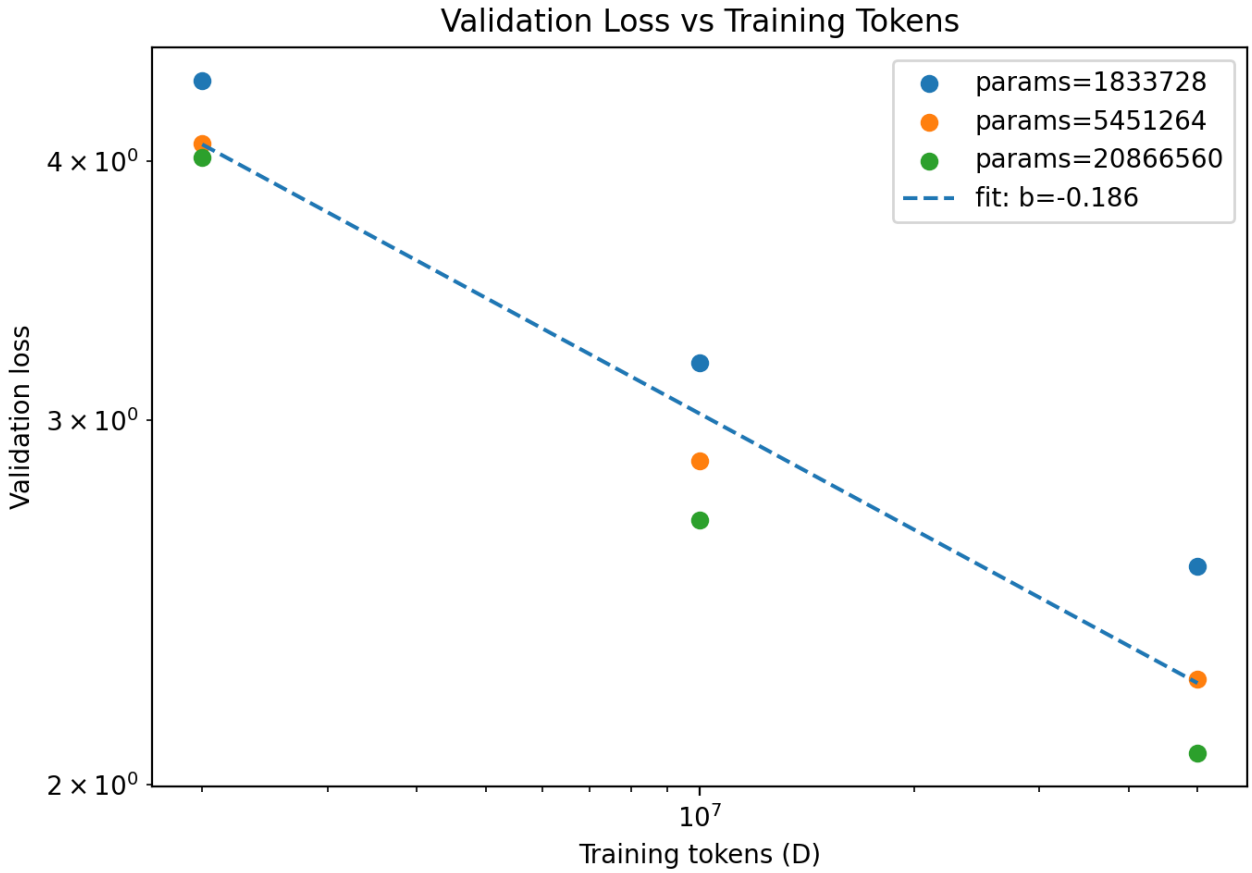
- **data scaling** showed the clearest and most stable empirical relationship,
- **compute scaling** also showed a strong monotonic trend,
- **parameter scaling**, when viewed in isolation as a single 1D curve across all runs, was comparatively weak.

Relation to the Figures

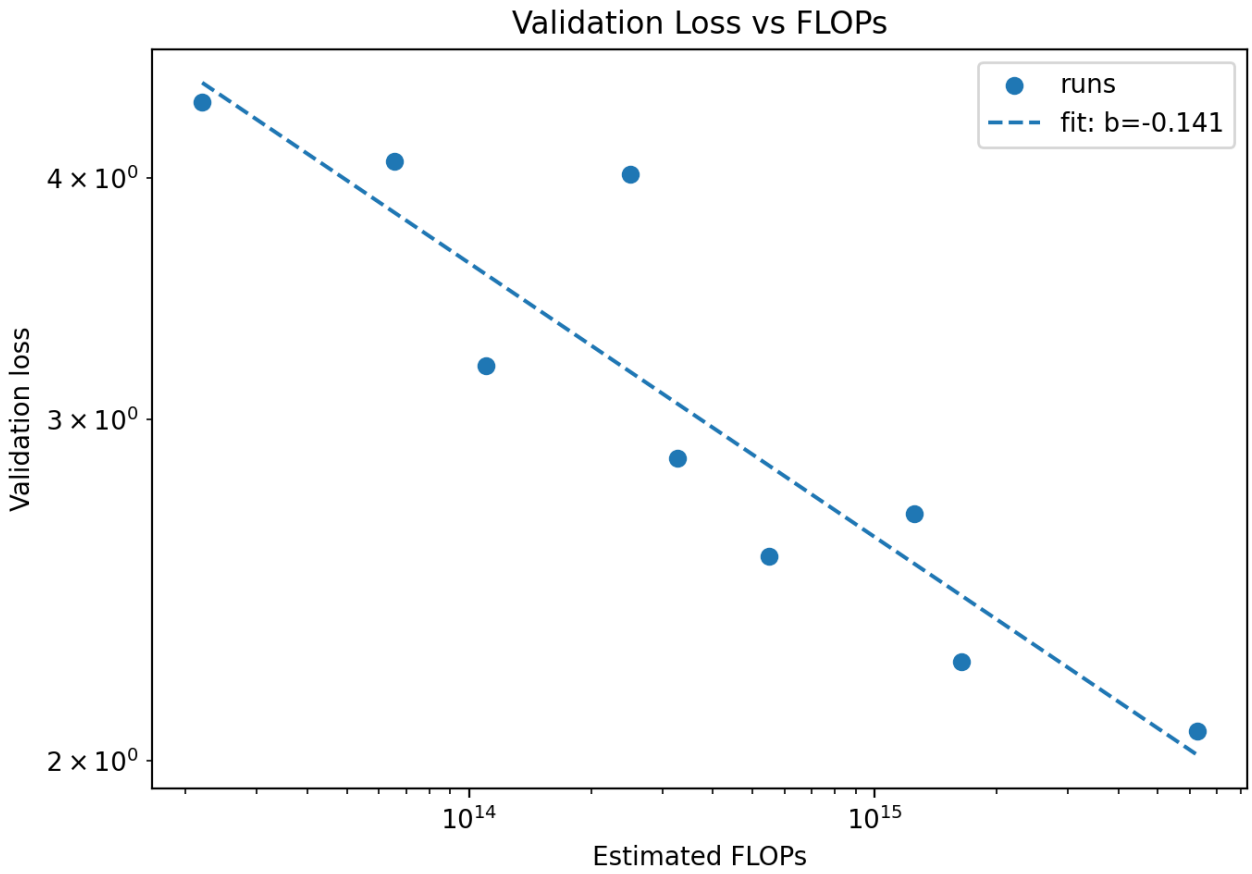
- **Figure 7** shows that the parameter-only fit is weak and that the runs do not collapse cleanly to a single curve when token budget varies. **Figure file:** `reports/figures/loss_vs_params.png`



- **Figure 8** shows the clearest power-law behavior, with loss decreasing consistently as training tokens increase. **Figure file:** `reports/figures/loss_vs_tokens.png`



- **Figure 9** shows a similarly strong compute trend, though slightly noisier than the token-based fit. **Figure file:** `reports/figures/loss_vs_flops.png`



Scope of the Fitting

These fits are intentionally simple empirical 1D fits motivated by prior scaling-law work, but they are not full joint fits of the type used in larger studies (Kaplan et al., 2020; Hoffmann et al., 2022). They are useful for visualizing the main trends in the sweep and for comparing them with the scaling-law literature. In particular, they are not joint fits of the form:

$$L(N, D) = E + A N^{-\alpha} + B D^{-\beta}$$

Therefore, the exponents reported here should be interpreted as descriptive summaries of the observed trends in this small-scale experimental setup rather than as definitive estimates of universal scaling constants.

Figures

- **Figure 7.** Validation loss vs parameters
  Figure file: `reports/figures/loss_vs_params.png`
- **Figure 8.** Validation loss vs training tokens
  Figure file: `reports/figures/loss_vs_tokens.png`
- **Figure 9.** Validation loss vs FLOPs
  Figure file: `reports/figures/loss_vs_flops.png`

---

9. Compute-Optimal Frontier

To complement the scaling-law fits, we computed an **empirical compute frontier** from the 9-run experimental grid. The frontier was defined directly from the observed runs: after sorting runs by increasing estimated FLOPs, a run was placed on the frontier if it achieved a strictly lower validation loss than all cheaper runs.

This gives a practical notion of **compute-optimality** within our sweep: for a given compute budget, which (N, D) allocation achieved the best observed validation loss?

Empirical Frontier Runs

The following runs appeared on the empirical frontier:

Run name	Parameters	Training tokens	FLOPs	Validation loss
model_1m_tok_2000000	1,833,728	1,999,488	21,999,102,787,584	4.3757
model_5m_tok_2000000	5,451,264	1,999,488	65,398,421,716,992	4.0800
model_1m_tok_10000000	1,833,728	9,999,472	110,017,870,749,696	3.1986
model_5m_tok_10000000	5,451,264	9,999,472	327,058,570,395,648	2.8659
model_1m_tok_50000000	1,833,728	49,999,392	550,111,710,560,256	2.5505
model_5m_tok_50000000	5,451,264	49,999,392	1,635,359,313,788,928	2.2496
model_20m_tok_50000000	20,866,560	49,999,392	6,259,891,878,789,120	2.0714

Interpretation

The frontier shows that **larger models were not always compute-optimal at lower token budgets**. In particular, the 20M-parameter model only appeared on the frontier at the **highest token budget (50M tokens)**. The runs:

- `model_20m_tok_2000000`

- `model_20m_tok_10000000`

did not appear on the frontier, because cheaper alternatives achieved lower validation loss before those runs became competitive.

This is an important result. It suggests that in this small-scale setup, allocating compute to a **moderate model trained on more data** was often better than allocating the same general budget to a much larger but less well-trained model.

Another useful observation is that the frontier alternates between the 1M and 5M models over a broad range of compute budgets. This means that the best allocation of compute was not "always maximize model size," but rather depended on the interaction between:

- model size,
- token budget,
- and total compute.

Main Takeaway

The empirical frontier supports a Chinchilla-style interpretation of the sweep: compute-optimal training depends on the balance between model size and data, not on model size alone (Hoffmann et al., 2022). In our runs, the best low-to-mid budget choices were often smaller or medium-sized models trained on more tokens, while the largest model became competitive only when paired with the largest training-token budget.

Relation to the Figure

Figure 10 shows the empirical compute frontier in FLOPs-loss space. The frontier traces the best observed validation loss reachable at increasing compute budgets and highlights which runs are dominated and which are compute-optimal within the experimental grid. **Figure file:** `reports/figures/compute_frontier.png`

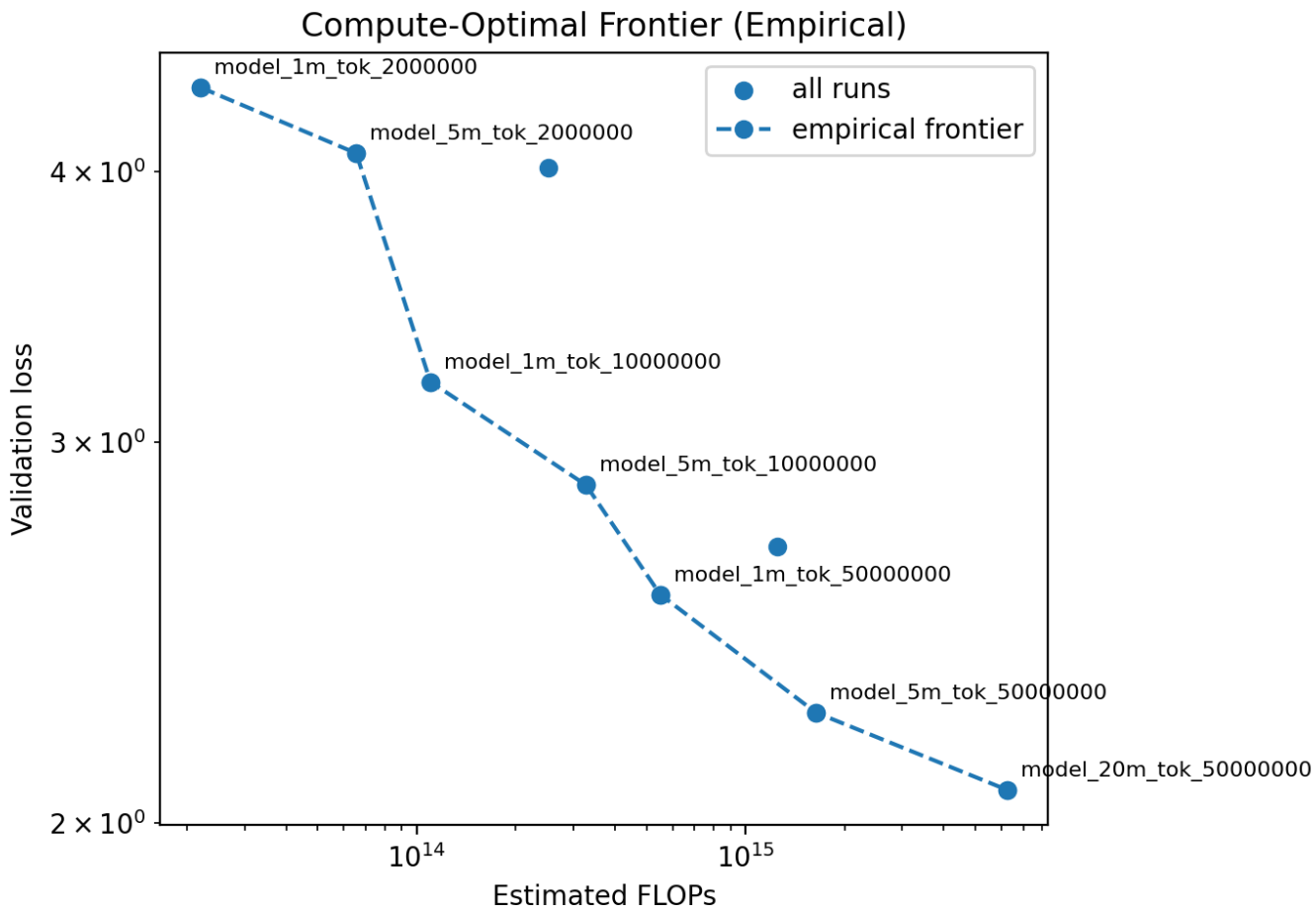


Figure and Table

- **Figure 10.** Empirical compute frontier
  Figure file: `reports/figures/compute_frontier.png`
- **Table 5.** Frontier runs
  Table file: `reports/tables/compute_frontier.csv`

---

10. Discussion

The results show a clear empirical scaling trend in this small-scale setup. The strongest signal appears along the data axis: increasing training tokens consistently reduces validation loss, with a strong log-log fit. Compute also exhibits a strong monotonic relationship with validation loss. By contrast, a simple 1D parameter-only fit is much weaker, which is expected because model size and token budget vary simultaneously across the sweep.

The empirical compute frontier further shows that larger models are not automatically compute-optimal at lower token budgets. In this sweep, the largest model only became frontier-optimal at the highest token budget, while smaller and medium-sized models dominated parts of the low-to-mid compute regime. This supports the broader idea that useful scaling analysis should consider the balance between model size and data, not model size alone.

---

11. Limitations

This study has several important limitations that should be considered when interpreting the reported scaling trends.

Small-Scale Experimental Regime

Although the experimental design was inspired by large-scale scaling-law studies, all runs were conducted in a much smaller compute regime. The models, token budgets, and total FLOP budgets were intentionally scaled down so that the full sweep could be executed on a consumer GPU. As a result, the fitted exponents should be interpreted as **small-scale empirical trends** rather than precise estimates of large-scale language-model scaling laws.

Limited Grid Size

The final sweep contains **9 runs** in total, corresponding to a 3×3 grid over model size and token budget. This is sufficient to reveal clear monotonic trends, but still too small for highly stable estimation of universal scaling exponents. In particular, sparse coverage of the (N, D) space makes the analysis more sensitive to local design choices.

Single Dataset and Single Domain

All experiments were run on `alexliap/tinystories-gr`, a Greek translation of TinyStories. This corpus is valuable because it provides a large Greek-only training signal, but it is also domain-constrained: the language is simple, repetitive, and targeted toward children's stories. Therefore, the measured scaling behavior may not generalize directly to broader Greek corpora or more diverse language-modeling settings.

Single Seed per Run

Each experiment was run with **one random seed only**. This means the results do not capture run-to-run variance due to initialization, data ordering, or optimization noise. Repeating each run with multiple seeds would provide more robust estimates and confidence intervals for both the measured losses and the fitted scaling exponents.

Simplified Fitting Procedure

The scaling analysis used simple **1D log-log power-law fits** for:

- loss vs parameters,
- loss vs training tokens,
- loss vs FLOPs.

These fits are useful descriptive summaries, but they are not equivalent to a full joint scaling-law fit of the form (L(N, D)). This limitation is especially visible in the weak parameter-only fit, where model size and token budget vary simultaneously and cannot be fully disentangled by a single 1D regression.

Fixed Context Length and Shared Training Protocol

All runs used the same tokenizer, the same context length (**128**), and the same general optimizer/training setup. This was the correct choice for experimental control, but it also means that the reported scaling behavior is conditional on this specific setup. Different context lengths, optimization schedules, or regularization choices could shift the observed exponents and frontier.

Empirical Frontier, Not a Theoretical Optimum

The compute frontier reported in this work is an **empirical frontier over the observed runs**, not a closed-form optimum derived from a continuous fitted objective. It identifies the best configurations within the tested grid, but it does not guarantee that those configurations are globally optimal outside the sampled experiment space.

Practical Compute Constraints

The experiments were executed under practical hardware constraints on a single NVIDIA RTX 3060 (12GB VRAM). Approximate sweep runtimes were:

- **small model sweep:** ~7-8 minutes
- **medium model sweep:** ~25–30 minutes
- **large model sweep:** ~50–60 minutes

These constraints influenced the final grid design, the number of runs, and the choice to keep the study compact and fully reproducible.

What We Would Improve With More Compute

Given more compute, the most important next improvements would be:

1. run multiple seeds per configuration,
2. increase the number of model sizes and token budgets,
3. fit a joint (L(N, D)) scaling law rather than only 1D projections,
4. evaluate on broader Greek-language corpora beyond TinyStories-GR,
5. explore larger context lengths and longer training schedules.

---

12. Conclusion

We built a complete and reproducible Greek-language pretraining pipeline from scratch, including tokenizer training, data preparation, model training, structured sweeps, training-curve generation, scaling-law fitting, and empirical compute-frontier analysis.

The main conclusions are:

1. validation loss decreases clearly with additional training tokens,
2. validation loss also improves systematically with increased compute,
3. the strongest empirical signal in this setup comes from data and compute scaling,

4. compute-optimal configurations depend on the interaction between model size and token budget.

Although the experiments were intentionally small-scale, they reproduced the qualitative behavior expected from scaling-law studies and provided a practical end-to-end framework for controlled pretraining experiments in Greek.

Appendix A. GPT-Style Architecture and Code Mapping

A1. Model Architecture Overview

We implemented a **GPT-style decoder-only Transformer language model** from scratch in PyTorch. The model is designed for **causal language modeling**, meaning that at each position it predicts the **next token** given only the tokens to its left.

The architecture follows the standard high-level GPT pattern:

1. **Input token ids** are mapped to dense vector representations through a learned **token embedding** layer.
2. A learned **positional embedding** is added so that the model can distinguish token order inside the sequence.
3. The resulting sequence representation is passed through a **stack of Transformer blocks**.
4. A final **LayerNorm** is applied to the hidden states.
5. A final **language modeling head** projects hidden states to vocabulary logits, from which next-token probabilities are computed.

Input Representation

The model operates on tokenized sequences of shape **(B, T)**, where:

- **B** is the batch size
- **T** is the sequence length

Each token id is first converted into a learned embedding vector through:

- `self.token_embedding`

In parallel, each sequence position is represented by a learned positional embedding through:

- `self.position_embedding`

These two embeddings are summed:

```
x = token_embedding + positional_embedding
```

This sum forms the initial input representation for the Transformer stack.

A1.1. Transformer Stack

The core of the model is a stack of `n_layer` **Transformer blocks**:

- `self.blocks = nn.ModuleList([Block(config) for _ in range(config.n_layers)])`

For example, in `configs/model_small.yaml`, the setting:

```
n_layer: 4
```

means that the input sequence passes through **4 Transformer blocks sequentially**.

Each **Block** is a **pre-LayerNorm Transformer block** and contains:

1. Causal Self-Attention

Implemented in:

- `CausalSelfAttention`

This module allows each token to attend only to:

- itself
- previous tokens

but **not future tokens**.

This behavior is enforced by:

```
F.scaled_dot_product_attention(..., is_causal=True)
```

which makes the model suitable for **autoregressive next-token prediction**.

2. Feed-Forward Network (MLP)

Implemented in:

- `MLP`

This is the standard Transformer feed-forward sublayer:

```
Linear → GELU → Linear → Dropout
```

It transforms the hidden representation at each position independently.

3. Residual Connections and LayerNorm

Implemented in:

- `Block`

Each block applies:

```
x = x + attention(LayerNorm(x))
x = x + mlp(LayerNorm(x))
```

So every block contains:

- one LayerNorm before attention
- one residual connection around attention
- one LayerNorm before the MLP

- one residual connection around the MLP

This is the standard **pre-LN Transformer design**, which improves training stability.

A1.2. Final Layers

After all Transformer blocks, the model applies a final normalization layer:

- `self.ln_f`

Then the hidden states are projected to vocabulary logits using:

- `self.lm_head`

This produces output of shape:

- **(B, T, vocab_size)**

so that for every position in the sequence, the model predicts a probability distribution over the vocabulary.

If `tie_embeddings=True`, the output projection shares weights with the input token embedding table:

```
self.lm_head.weight = self.token_embedding.weight
```

This is a common language-modeling design choice that reduces parameter count and keeps input/output token representations aligned.

A1.3. Forward Pass Summary

The full forward path is:

```
token ids
→ token embeddings
+ positional embeddings
→ Transformer block 1
→ Transformer block 2
→ ...
→ Transformer block N
→ final LayerNorm
→ language modeling head
→ vocabulary logits
```

For the small model configuration with `n_layer: 4`, this becomes:

```
token ids
→ token embeddings + positional embeddings
→ block 1
→ block 2
→ block 3
→ block 4
→ final LayerNorm
→ lm head
```

A1.4. Training Objective

The model is trained with **token-level cross-entropy loss**. When target token ids are provided, the forward method computes:

```
F.cross_entropy(logits.reshape(-1, logits.size(-1)), targets.reshape(-1))
```

This means the model is optimized to predict the correct next token at each position in the sequence.

A2. GPT Design Patterns Followed

To ensure that the implemented model follows the **GPT architecture pattern** rather than a generic Transformer, we adopted the following design choices.

A2.1. Decoder-only Transformer design

The implemented model is **decoder-only**, not encoder-decoder and not encoder-only.

This means the model is designed for **autoregressive language modeling**: it processes a sequence from left to right and predicts the next token at each position.

This design is reflected in the fact that the model consists of:

- token embeddings
- positional embeddings
- a stack of Transformer decoder blocks
- a final language modeling head

without any encoder stack or cross-attention mechanism.

A2.2. Causal self-attention

A defining GPT pattern is that attention must be **causal**.

In practice, this means that each token can attend only to:

- itself
- previous tokens

but not to future tokens.

This property is implemented in the attention layer through:

```
F.scaled_dot_product_attention(..., is_causal=True)
```

This is one of the most important reasons why the model is GPT-style: it enforces the left-to-right prediction constraint required for next-token generation.

A2.3. Autoregressive next-token prediction objective

GPT models are trained to predict the **next token** in a sequence.

This pattern is followed in the implementation because the model outputs logits over the vocabulary for every sequence position, and training uses token-level cross-entropy loss against shifted targets.

So the model is not trained for:

- masked token reconstruction
- sequence classification
- encoder-decoder generation

It is trained specifically for **autoregressive language modeling**.

A2.4. Repeated Transformer block structure

Another GPT pattern is the use of a repeated stack of identical Transformer blocks.

In this implementation, the model depth is controlled by:

- `n_layer`

and the blocks are created as:

```
self.blocks = nn.ModuleList([Block(config) for _ in range(config.n_layers)])
```

Each block follows the standard GPT/Transformer decoder pattern:

- self-attention sublayer
- feed-forward sublayer
- residual connections
- normalization

This repeated-block design is central to GPT-style scaling.

A2.5. Pre-LayerNorm residual architecture

The implemented block uses a **pre-LN** formulation:

```
x = x + self.attn(self.ln_1(x))
x = x + self.mlp(self.ln_2(x))
```

This means normalization is applied **before** each sublayer.

This is a widely used modern GPT-style training pattern because it improves optimization stability, especially as models become deeper.

A2.6. Learned token embeddings and learned positional embeddings

The model uses:

- learned token embeddings
- learned positional embeddings

rather than sinusoidal positional encodings or handcrafted input features.

This is consistent with the GPT family design, where both token identity and token position are learned directly from data.

A2.7. Language modeling head over the vocabulary

The final output layer maps hidden states to vocabulary logits through a linear projection:

- `self.lm_head`

This is the standard GPT output pattern: for each sequence position, the model produces a distribution over the full vocabulary and selects the most likely next token.

A2.8. Optional weight tying between input and output embeddings

The implementation supports:

- `tie_embeddings: true`

When enabled, the output projection weights are shared with the input embedding table:

```
self.lm_head.weight = self.token_embedding.weight
```

This is a common GPT-style optimization that:

- reduces parameter count
- improves parameter efficiency
- keeps input and output token representations aligned

A2.9. Random initialization and training from scratch

The model does not import pretrained GPT weights.

Instead, it initializes weights from scratch using standard Gaussian initialization for linear and embedding layers.

This is important for the assignment because the requirement is to train models **from scratch**, not to fine-tune a pretrained model.

Summary

The model follows the GPT architecture pattern because it combines:

- a **decoder-only Transformer structure**
- **causal self-attention**
- **autoregressive next-token prediction**
- a **stack of repeated Transformer blocks**
- **pre-LayerNorm residual design**
- **learned embeddings**
- a **vocabulary projection head**
- optional **embedding weight tying**

Together, these design decisions make the implementation a **GPT-style causal language model**, even though it is implemented directly in PyTorch rather than imported from a pretrained GPT library.

A3. Architecture-to-Code Mapping

This section maps each major architectural component of the implemented GPT-style model to the corresponding code elements in `src/scaling_laws/models/gpt.py`.

A3.1. Model configuration

The overall model shape is defined by the `GPTConfig` dataclass.

Relevant code:

- `GPTConfig`
- `GPTConfig.from_dict(...)`

This component is responsible for translating the YAML configuration into model hyperparameters such as:

- vocabulary size
- context length
- embedding dimension
- number of layers
- number of attention heads
- dropout
- weight tying

In particular, `from_dict(...)` maps the project configuration keys:

- `n_embd`
- `n_layer`
- `n_head`

to the internal model fields:

- `d_model`
- `n_layers`
- `n_heads`

A3.2. Token embedding layer

The token embedding layer is implemented as:

- `self.token_embedding = nn.Embedding(config.vocab_size, config.d_model)`

This layer converts discrete token ids into dense continuous vectors of dimension `d_model`.

A3.3. Positional embedding layer

The positional embedding layer is implemented as:

- `self.position_embedding = nn.Embedding(config.context_length, config.d_model)`

This layer provides a learned representation of token positions in the sequence.

A3.4. Input representation construction

The input representation is formed in the `forward(...)` method of `GPTLanguageModel` through:

- `tok_emb = self.token_embedding(idx)`

- `pos_emb = self.position_embedding(positions)`
- `x = self.dropout(tok_emb + pos_emb)`

This is the point where token identity and token position are combined before entering the Transformer stack.

A3.5. Transformer block stack

The repeated Transformer blocks are implemented as:

- `self.blocks = nn.ModuleList([Block(config) for _ in range(config.n_layers)])`

This means the model depth is determined directly by `config.n_layers`.

For example:

- if `n_layer = 4`, then the model contains 4 stacked Transformer blocks

A3.6. Transformer block structure

A single Transformer block is implemented in:

- `class Block(nn.Module)`

Inside `Block`, the two main sublayers are:

- `self.attn = CausalSelfAttention(config)`
- `self.mlp = MLP(config)`

and the two normalization layers are:

- `self.ln_1 = nn.LayerNorm(config.d_model)`
- `self.ln_2 = nn.LayerNorm(config.d_model)`

The forward pass of one block is:

```
x = x + self.attn(self.ln_1(x))
x = x + self.mlp(self.ln_2(x))
```

This code implements:

- LayerNorm before attention
- residual connection around attention
- LayerNorm before the MLP
- residual connection around the MLP

A3.7. Causal self-attention

The self-attention mechanism is implemented in:

- `class CausalSelfAttention(nn.Module)`

Key architectural elements in this class are:

Query / Key / Value projection

- `self.qkv_proj = nn.Linear(config.d_model, 3 * config.d_model, bias=config.bias)`

This creates Q, K, and V in one projection for efficiency.

Multi-head split

Inside `forward(...)`, the tensor is reshaped into multiple heads:

- `q = q.view(...).transpose(...)`
- `k = k.view(...).transpose(...)`
- `v = v.view(...).transpose(...)`

Causal masking

The GPT-style causal constraint is implemented by:

```
F.scaled_dot_product_attention(
    q,
    k,
    v,
    attn_mask=None,
    dropout_p=self.dropout if self.training else 0.0,
    is_causal=True,
)
```

The argument:

- `is_causal=True`

is the key implementation detail that makes this attention layer autoregressive.

Output projection

After merging heads back together, attention output is projected through:

- `self.out_proj = nn.Linear(config.d_model, config.d_model, bias=config.bias)`

A3.8. Feed-forward network (MLP)

The feed-forward network inside each block is implemented in:

- `class MLP(nn.Module)`

Its layers are:

- `self.fc = nn.Linear(config.d_model, hidden_dim, bias=config.bias)`
- `self.proj = nn.Linear(hidden_dim, config.d_model, bias=config.bias)`

and the forward path is:

```
x = self.fc(x)
x = F.gelu(x)
x = self.proj(x)
x = self.dropout(x)
```

This corresponds to the standard Transformer MLP pattern:

- linear expansion
- GELU activation
- projection back to model dimension
- dropout

A3.9. Final layer normalization

After the full Transformer stack, the model applies a final normalization layer:

- `self.ln_f = nn.LayerNorm(config.d_model)`

This is used in the main forward pass as:

- `x = self.ln_f(x)`

A3.10. Language modeling head

The final projection to vocabulary logits is implemented as:

- `self.lm_head = nn.Linear(config.d_model, config.vocab_size, bias=False)`

This produces logits of shape:

- `(B, T, vocab_size)`

so that each sequence position predicts the next token distribution over the vocabulary.

A3.11. Weight tying

Optional weight tying is implemented through:

```
if config.tie_embeddings:
    self.lm_head.weight = self.token_embedding.weight
```

This connects:

- input token embeddings
- output vocabulary projection

using the same weight matrix.

A3.12. Weight initialization

Model initialization is implemented in:

- `def _init_weights(self, module):`

This function initializes:

- linear layers with Gaussian weights
- embedding layers with Gaussian weights
- biases to zero

The initialization is applied through:

- `self.apply(self._init_weights)`

A3.13. Forward pass

The main forward computation is implemented in:

- `def forward(self, idx, targets=None):`

This method:

1. checks that sequence length does not exceed the configured context length
2. builds position indices
3. looks up token and positional embeddings
4. sums them
5. passes the result through all Transformer blocks
6. applies final layer normalization
7. projects to vocabulary logits
8. optionally computes cross-entropy loss if `targets` are provided

A3.14. Loss computation

The training loss is implemented directly in the forward method through:

```
loss = F.cross_entropy(
    logits.reshape(-1, logits.size(-1)),
    targets.reshape(-1),
)
```

This corresponds to standard token-level next-token prediction loss.

A3.15. Model construction helper

The convenience entrypoint for constructing the model is:

- `build_model(model_cfg: dict, vocab_size: int)`

This function:

1. builds a `GPTConfig`
2. instantiates `GPTLanguageModel`
3. returns the model object

This is the main interface used by the training code.

Short Mapping Summary

- **model hyperparameters** → `GPTConfig`
- **config-file translation** → `GPTConfig.from_dict(...)`
- **token embeddings** → `self.token_embedding`
- **positional embeddings** → `self.position_embedding`
- **Transformer stack** → `self.blocks`

- **one Transformer block** → `Block`
- **causal attention** → `CausalSelfAttention`
- **feed-forward network** → `MLP`
- **final normalization** → `self.ln_f`
- **vocabulary projection head** → `self.lm_head`
- **weight tying** → `self.lm_head.weight = self.token_embedding.weight`
- **loss computation** → `F.cross_entropy(...)`
- **model builder** → `build_model(...)`

Appendix B. Tokenizer Profiling Metric Definitions

B1. Vocabulary Profiling Metrics

Let V denote the tokenizer vocabulary size.

Fertility $$\text{fertility} = \frac{\text{total number of subword tokens}}{\text{total number of whitespace-delimited words}}$$

Lower fertility indicates less fragmentation.

Embedding share For the smallest model with hidden size d and total trainable parameters N, the approximate embedding share is:

$$\text{embedding share} = \frac{V \cdot d}{N}$$

when input and output embeddings are tied. If untied embeddings are used, the numerator becomes approximately $2 \cdot V \cdot d$.

Active vocabulary ratio $$\text{active vocab ratio} = \frac{\#\{t \in V : \text{count}(t) \ge k\}}{|V|}$$

where k is a minimum usage threshold on the profiling subset.

References

- Kaplan, J., et al. (2020). *Scaling Laws for Neural Language Models.*
- Hoffmann, J., et al. (2022). *Training Compute-Optimal Large Language Models.*
- Rust, P., et al. (2021). *How Good is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models.*